Analyse v16 → v17 — Audit, référence de performance et plan de tests

Fichier : analyse_problemes_v16_v17.md · **Dossier :** wiki (racine) · `md/optimisation/` (copie) **Branche :** master (wiki) · **Auteur :** hprzeta · **MAJ :** 2026-10-15 **Statut :** audit + benchmark de référence terminés ; **aucune modification de code** à ce stade.

Règle directrice (consigne V17). Ne pas confondre *rapidité*, *précision d'une racine* et *complétude d'un calcul*. Le but de V17 est le rapport performance/fiabilité sur PC1 et PC2, pas un score de vitesse isolé.

Légende des statuts : [**mesuré**] observé ce jour · [**lu**] constaté dans le code ou un log · [**hypothèse**] non démontré · [**conjecture**] énoncé mathématique non prouvé.

---

1. Sources et références

Élément	Valeur
Commit de départ de la session	`36c5b83` (`Riemann_Lab_IA`)
Commit V16 (illinois_arb.c, précision fixe)	`00abe5c` (08/08/2026) ; `1b83cdb` sur `Riemann_Lab_IA`
Persistance CSV avant rescan	`97e3d1f` (18/08/2026)
Fichiers exécutés	`compute_zeros_v16.py`, `c_modules/illinois_arb.so`, `c_modules/scan_arb.so`
Bibliothèque FLINT chargée	`libflint-24205715.so.21.0.0` (FLINT 3.3.1, `python-flint` 0.8.0)
Headers de compilation	`c_modules/flint-headers-3.3.1/` (vendorisés, 23 fichiers)
Rapport de transition V15→V16	pas de fichier distinct : §5 de [[analyse_problemes_v1_v16]]

Lien avec les analyses existantes : [[analyse_problemes_v1_v16]], [[Plancher-Hardware-Architecture]], [[Optimisation-Pourquoi-le-calcul-va-vite]], [[Analyse-Deficit-Zeros-Grille-20260906]], [[analyse_math_deficit_2026-08-18]].

---

2. Référence de performance V16 (T = 100 000) [mesuré]

Matériel PC1 : Intel i7-7500U (2 cœurs, 4 threads, AVX2/FMA/BMI2), 16 Go de RAM, 16 Go de swap, Ubuntu 24.04, gcc 13.3 (`-O3 -march=native`), secteur branché, scripts turbo actifs (gouverneur `performance`), 8 workers, STEP adaptatif du code, `MARGE_SECURITE` du code.

Commande : `scripts/zeta_run.sh 100000` (6 répétitions, deux lots : 12h51–13h00 et 14h33–14h43).

Run	Durée du run principal	Zéros	Fallback mpmath	LMFDB	Turing
1	93,6 s	138 069	569 (0,4 %)	20/20	COMPLET
2	87,9 s	138 069	569	20/20	COMPLET
3	88,8 s	138 069	569	20/20	COMPLET
4	92,3 s	138 069	569	20/20	COMPLET
5	104,1 s	138 069	569	20/20	COMPLET
6	115,1 s	138 069	569	20/20	COMPLET

- **Durée :** médiane 93 s, minimum 87,9 s, maximum 115,1 s. Référence historique documentée : 98,1 s.
- **Dispersion :** 87,9–93,6 s pour le premier lot ; la durée **augmente** dans le second lot (92 → 104 → 115 s). Cause non établie. [**hypothèse**] : échauffement du CPU (pause de 20 s entre runs). Les températures n'ont pas été relevées : à vérifier avant d'interpréter un gain inférieur à 20 %.
- **Durée totale avec rescan et Turing :** de 116 s à 138 s (écart avec le run principal : rescan ciblé, validation Turing, visualisation).
- **Listes de zéros :** les 6 listes sont **identiques bit à bit** (SHA-256 identique après arrondi à 9 décimales, écart maximal entre runs : 0). Elles sont donc reproductibles, pas seulement de même taille.
- **Mémoire :** RSS cumulé des 8 workers **≈ 2,04 Go** au maximum (≈ 255 Mo par worker), **swap utilisé : 0**. Le champ swap de l'échantillonneur était vide (localisation de `free`) ; la valeur « 0 » vient d'un contrôle de `free` en fin de run, pas d'un échantillonnage continu.
- **Plus petit écart consécutif de la liste :** 0,0147 (n'est pas un test de complétude).

Chaque run enregistre un CSV principal avant le rescan (correctif `97e3d1f`) : les 6 dossiers `calculs/v16_T100000_20261015_*` en contiennent un.

Limite. T = 100 000 ne montre ni les phénomènes de grand t ni le déséquilibre de charge du run T = 5M. Le run V16 à T = 5M (10 016 297 zéros sur 10 016 474, 21 h 38) a été **perdu** dans un crash le 18/08 (CSV jamais écrit) : il n'existe aucune référence V16 à cette échelle.

---

3. Écarts relevés par l'audit

3.1 Déduplication à `tolerance=0.01` [hypothèse confirmée par T1b sur un intervalle]

`dedupliquer()` (`parallel_scanner.py:148`) fusionne deux zéros distants de moins de 0,01. Or l'écart normalisé de deux zéros consécutifs peut être très petit à grand t.

- **Mesure [mesuré]** sur le CSV du run v13 T = 5M (10 016 377 zéros) : **0** écart ≤ 0,0100 et **9** écarts dans [0,0100 ; 0,0105[. Les plus petits : 0,010094 (t = 3 044 156), 0,010096 (t = 1 573 834). Une coupure nette à 0,01 est la signature d'un seuil de déduplication.
- **Ordre de grandeur [conjecture de Montgomery/GUE]** : $p(s) \sim \frac{\pi^2}{3} s^2$ pour $s \to 0$, donc $M_1(h) \approx \frac{\pi^2}{9} \alpha^3 N$ avec $\alpha = h/\bar{\delta}(t)$. Pour $h = 0{,}01$, $t \approx 3 \times 10^6$ ($\bar{\delta} \approx 0{,}48$) et $N \approx 1{,}25 \times 10^6$ par segment : $\alpha \approx 0{,}021$ et $M_1 \approx 12$ zéros par segment.
- **Comparaison au run V16 T = 5M** [lu] : déficits par segment 51, 49, 16, 13, 10, 9, 14, 14. Les six segments à grand t (9 à 16) sont **compatibles avec ≈ 12**. Les segments 0 et 1

montrent un excès (≈ +38 chacun) qui reste compatible avec la fragilité de Riemann-Siegel à bas t (cf. [[Analyse-Deficit-Zeros-Grille-20260906]]).
- **Pourquoi l'analyse du 06/09 ne le voit pas :** elle calcule M_1 avec le pas de grille ($h = 0{,}00157 \Rightarrow M_1 \approx 0{,}08$), pas avec la tolérance de déduplication (0,01), 6,4 fois plus grande (le terme en h^3 fait un facteur ≈ 260).
- **Test proposé :** reconstituer la liste brute (avant déduplication), ne dédupliquer que les zéros de la zone de recouvrement entre segments, et comparer. Ne change aucune constante de calibration.
- **Statut :** hypothèse confirmée sur un intervalle par T1b (ci-dessous). Ne pas corriger l'explication historique avant extension du test (T=5M complet).

Résultat T1b — scan V16 ciblé [4 900 000 ; 4 986 000], 15/10/2026 (33,0 min, 8 workers, code de sortie 0, sans rescan ; `calculs/v16_T4900000_4986000_T1b/`).

- **Mesuré — zéros :** 185 821 zéros bruts (concaténation des 8 `checkpoints/worker_*.csv`), 185 810 après déduplication : 11 supprimés. Le CSV final et `_principal.csv` sont identiques.
- **Mesuré — 7 doublons artificiels :** écart exactement 0, une même racine trouvée par deux workers voisins (zone de recouvrement de 0,5), une par frontière de segment.
- **Mesuré — 4 paires réelles supprimées :** écarts 3,03e-3 (t = 4 926 490,100), 7,17e-3 (4 927 353,414), 7,24e-3 (4 958 530,097), 7,79e-3 (4 933 944,885). Confirmées par mpmath (`siegelz`, dps = 30) : Z change de signe de part et d'autre d'un point intermédiaire de signe opposé aux deux points extérieurs (signes $- + -$, $- - +$, $- + -$, $- + -$), et les deux racines, raffinées indépendamment, restent distinctes (écarts confirmés 3,028e-3 ; 7,169e-3 ; 7,241e-3 ; 7,787e-3). Zéros distincts réels : 185 814.
- **Mesuré — comptage :** $N(b) - N(a) = 185\,813{,}56$ (Backlund, θ asymptotique, facteur e dans $T/2\pi e$, $S(T)$ ignoré). Déficit **+3,56** avant correction, **−0,44** après ajout des 4 zéros. Le nombre attendu n'a pas été modifié ; `_n_zeros_expected` (Weyl tronqué) n'a pas été utilisé.
- **Mesuré — motif de v13 non reproduit :** dans le CSV dédupliqué, 0 écart ≤ 0,0100 et 0 dans [0,0100 ; 0,0105[(minimum 0,011474). Dans le brut, aucun écart entre 0,0078 et 0,0115.
- **Déduit :** la validation Turing de v16 (« 9 800 393 manquants ») et le score LMFDB 0/20 sont des artefacts d'intervalle partiel (delta constant ≈ nombre de zéros avant 4,9M ; LMFDB comparé aux zéros n° 1 à 20).
- **Hypothèse :** la déduplication doit distinguer les doublons de frontière (écart ≈ 0) des paires réelles (écart ≥ 1e-3), par exemple par une tolérance beaucoup plus fine appliquée à la seule zone de recouvrement.
- **Limites :** un seul intervalle de 86 000 en t, pas T = 5M complet ; $S(T)$ non traité (incertitude de ±1 à 2 zéros sur le comptage, la preuve repose sur les signes mpmath) ; 4 cas, trop peu pour juger si le motif de v13 est une signature de coupure.

3.2 Critère d'arrêt et incertitude de Z(t) [lu]

Cinq grandeurs sont à distinguer (consigne, étape 3) :

Grandeur	État dans V16
Précision en bits (FLINT/ACB)	64 bits fixe (`PREC_BITS_ARB`)
Incertitude sur $Z(t)$	**non exploitée** : seul le milieu de la boule est lu (`arf_get_d(arb_midref(...))`), le rayon est ignoré
Erreur estimée sur la racine	**non calculée**

Grandeur	État dans V16
Tolérance du raffinement	`tol=1e-12`, appliquée à $\|Z\|$ (`illinois_arb.c:254`) **et** à $\|\delta\|$ (ligne 257)
Erreur observée vs référence	4,6e-11 (benchmark historique) ; 1,455e-11 en production à $t = 10^5$ (Handoff)

- Aucune erreur rigoureuse sur la racine n'est calculée : tout est observé, rien n'est encadré.
- **Cohérence 4,6e-11 vs 1e-12 :** pas de conclusion. Le 4,6e-11 compare à `mpmath` dps = 35 sur 300 brackets (deux zones de t). La grandeur est vraisemblablement une erreur d'**abscisse**, mais ce point doit être vérifié dans le script du benchmark.
- **Modèle de v15 mis en doute [hypothèse] :** il prédit $\varepsilon \approx C^2 t^{-5/2} \approx 9 \times 10^{-17}$ à $t = 10^5$ après un Newton. La dérivée vient de Z_{rs}, qui est biaisée : la convergence est alors linéaire (erreur ≈ biais × erreur relative de la dérivée), pas quadratique. Cela expliquerait 1,5e-11 au lieu de 1e-16. À confirmer par une mesure, puis à documenter dans [[analyse_problemes_v13_v15]].

3.3 Pertes silencieuses [lu]

- `except Exception: pass` (`compute_zeros_v16.py:285`) : un bracket qui lève une exception disparaît sans trace. Aucun compteur.
- **Fallback mpmath :** 569 sur 138 061 à T = 100k (0,4 %), 11 294 à T = 5M (0,1 %), dont 6 329 pour le seul worker 0 (0,5 %). Le fallback se déclenche quand la racine sort du bracket (`a-1e-10 <= zero <= b+1e-10`). Chaque cas indique un affinage qui n'a pas convergé dans l'intervalle ; la précision du résultat de repli (`arb_hardy_z` + `mpmath`) n'est pas vérifiée à part.

3.4 Déséquilibre de charge [lu]

Run T = 5M : durée du worker 0 = 33 924 s, du worker 7 = 77 908 s (moyenne 63 877 s). Les segments portent le même nombre de zéros, mais le coût d'un zéro croît en $\sqrt{t}$. Gain potentiel de l'équilibrage : environ 18 % sur la durée de mur. Levier indépendant de la précision.

3.5 Portabilité PC1 / PC2 [mesuré par ssh en lecture seule]

	PC1	PC2
CPU	i7-7500U, 2 c / 4 t	**Core 2 Duo E8400, 2 c / 2 t, 3,0 GHz**
Instructions	AVX2, FMA, BMI2	**pas d'AVX2, pas de FMA**
RAM	16 Go (+16 Go swap)	7,8 Go (≈ 6,5 Go libres, 11,5 Go swap)
OS	Ubuntu 24.04	Debian 12
Compilateur	gcc 13.3	gcc 12.2
FLINT	3.3.1 (bundle `python-flint`)	**2.9.0** (système) + Arb 2.23 ; `python-flint` 0.8.0 (FLINT 3.3.1) dans le Python système
Projet	v16	v13–v15, pas de v16, pas de git

Conséquences (corrigées le 15/10 après lecture de `zeta_sync_pc2.sh`) :

1. Les `.so` ne sont **jamais copiés** de PC1 vers PC2 : `zeta_sync_pc2.sh` les recompile sur PC2. Le risque d'instruction illégale (`-march=native`) n'existe donc pas dans le workflow normal. *(Une première version de ce rapport l'annonçait : erreur, corrigée.)*
2. `python-flint` 0.8.0 (FLINT 3.3.1) **est installé** sur PC2 dans le Python système. *(Première version : « absent », car seul `zeta_env` avait été regardé : erreur, corrigée.)*
3. **Vrai risque ABI :** `zeta_sync_pc2.sh` lie `illinois_arb.so` contre la `libflint-arb` système (Arb 2.23), correct pour V15 (`arb_fpwrap`). V16 exige les headers 3.3.1 vendorisés ; les lier contre Arb 2.23 risque une corruption mémoire silencieuse (voir §5.4 de [[analyse_problemes_v1_v16]]). Sur PC2, `flint-headers-3.3.1/` est absent et `compute_zeros_v16.py` aussi.
4. **Run distribué :** `scripts/zeta_distribute.py` (via `zeta-distribute`) appelle encore `compute_zeros_v15.py`, avec un ratio PC1/PC2 de 61,6/38,4 mesuré en V15 (822 et 512 z/s à T = 1000). Distribuer V16 suppose : lier contre la `libflint` 3.3.1 de `python-flint` sur PC2, synchroniser les headers vendorisés, changer le script cible, puis remesurer le ratio. La fusion PC1/PC2 utilise la même déduplication à 0,01 (§3.1).
5. Budget mémoire : 8 workers ≈ 2 Go sur PC1. PC2 a 2 cœurs sans hyperthreading : le nombre de workers (8 sur PC1) y est à mesurer, pas à reprendre.

3.6 Documentation obsolète ou contradictoire [lu]

Document	Écart
`Plancher-Hardware-Architecture` (MAJ 13/06)	« plancher i7 atteint à 8,8 min » alors que V16 mesure ≈ 90 s ; RAM 8 Go (16 Go depuis le 25/07) ; pied de page « 195 lignes » pour 271
`CLAUDE.md` racine	`MARGE_SECURITE=3.0` cité, 10.0 dans le code ; `T_SEUIL_ILLINOIS_C=300` obsolète depuis v12
`suivi_run_T5M_v16_20260816`	annonce « quelques heures » ; mesure : 21 h 38
`compute_zeros_v16.py` (en-tête de `main`)	affiche `T_SEUIL_PETIT_T=20`, le code utilise 65

---

4. Plan de tests (avant toute modification du code)

Une modification n'est acceptée que si le gain est reproductible, les racines identiques, la complétude préservée et la mémoire compatible avec PC2. Sinon V16 est conservé.

#	Test	Objet	Critère
T1	Dédup brute vs dédupliquée sur zone à grand t	Hypothèse §3.1	Nombre de paires < 0,01 retrouvées — **T1b fait le 15/10 : 4 paires réelles retrouvées sur [4,90M ; 4,986M]** ; T=5M complet à faire
T2	300 brackets × 5 grandeurs (rayon ACB, erreur de racine vs mpmath dps = 35, etc.)	§3.2	Erreur de racine sur l'abscisse, avec cas mal conditionnés

#	Test	Objet	Critère
T3	Profilage `perf` T = 100k + temps par worker	Goulots	Part de chaque étape, déséquilibre
T4	Non-régression des racines	Étape 4	Listes comparées (ordonnées), pas le seul comptage
T5	ABI : version de la `.so` chargée vs headers au chargement	Étape 5	Arrêt si les versions diffèrent
T6	PC2 : recompilation et exécution courte	Étape 7	Budget durée/RAM fixé avant lancement

Ordre : T1 → T2 → T3, puis seulement les leviers (équilibrage de charge, réutilisation des objets ACB, nombre de workers), un changement isolé à la fois, avec retour possible à V16.

---

5. Points restant à vérifier

- Origine de la dérive de durée du second lot de runs : sur T1b (run seul, 33 min), la fréquence a baissé de 3,5 à 3,0 GHz sous charge, stable à 3,0 GHz dès 16:17, avec un package à 73–76 °C (sous la limite de 100 °C). Cause (limite de puissance ou thermique) non établie ; le second lot du 15/10 avait été relevé sans températures.
- Identité exacte de la grandeur « 4,6e-11 » (script du benchmark de v16).
- Lien de `illinois_arb.so` V16 sur PC2 contre `python-flint` 3.3.1 (test T5/T6), sans toucher à la production.
- Statut du run T = 5M : à refaire avec persistance, après extension de T1b (T=5M complet).
- Le run T1b n'écrit rien avant la fin (log muet, `checkpoints/` vide pendant 33 min) : une perte de session fait tout perdre ; prévoir une progression dans le log.
- Corriger `CLAUDE.md` (marge, seuil) et `Plancher-Hardware-Architecture` **sur validation de hprzeta**.

---

analyse_problemes_v16_v17.md · wiki racine · branche master · hprzeta · MAJ 2026-10-15 · 232 lignes